

TDA / Algo3
1^{er} C 2025
2^{do} Parcial

Nombre y Apellido	L.U.	Nº Orden	Materia y turno	
			☐ TM ☐ TDA	☐ TN ☐ Algo3

Duración: 5 horas. Este examen es individual y a libro cerrado.

Los ejercicios 1 a 4 valen 5 puntos si se responden correctamente, -2 puntos si se responden incorrectamente. Cada uno tiene una única opción correcta. Marcar más de una opción se considera una respuesta incorrecta.

Los ejercicios 5 y 6 valen 10 puntos si se responden correctamente.

Los ejercicios 7 a 10 valen 15 puntos si se responden correctamente.

La nota de aprobación es de 60/100 puntos

Ejercicio 1: Un *coloreo* en un grafo es una asignación de colores a todos sus vértices de manera tal que si dos vértices son adyacentes entonces no reciben el mismo color. Un grafo de n vértices que es un ciclo es coloreable con 2 colores

☐ si y solo si n es potencia de 2 ☒ si y solo si n es par ☐ si y solo si $n \geq 2$ ☐ siempre

Ejercicio 2:Cuál es la complejidad temporal de sort en función de la longitud n del array de entrada?

```
def sort_rec(arr, i, j):
    if j < i+2:
        return
    if j == i+2:
        if arr[i+1] < arr[i]:
            swap(arr[i+1], arr[i])
        return
    n = j-i
    sort_rec(arr, i, j-n//3)
    sort_rec(arr, i+n//3, j)
    sort_rec(arr, i, j-n//3)

def sort(arr):
    sort_rec(arr, 0, len(arr))
```

☐ $\Theta(n \log_{3/2}(n))$ ☒ $\Theta(n^{\log_{3/2}(3)})$ ☐ $\Theta(n^{3/2 \cdot \log_3(n)})$ ☐ $\Theta(n^3)$

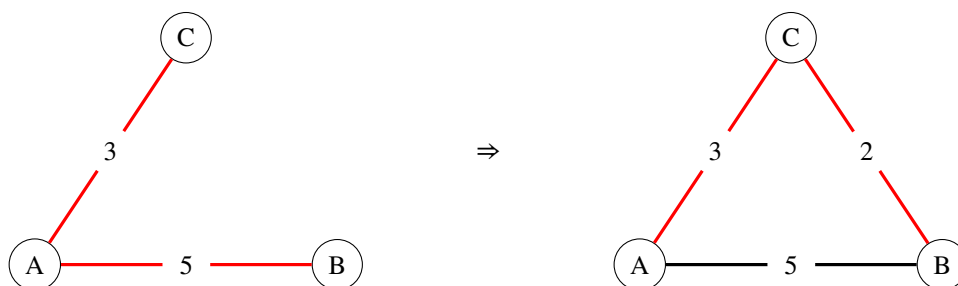
Ejercicio 3: Al aplicar el algoritmo de BFS en un grafo grilla (donde cada nodo es una celda, y está conectado con sus vecinos, que son 4 para las celdas internas, 2 para las esquinas y 3 para los bordes que no son esquinas). Si se ejecuta una búsqueda BFS comenzando en esquina (0,0) puede asegurarse que un vértice (i, j) se visita **después** que el vértice (2, 3) cuando

☐ $i > 2$ ☐ $j > 3$ ☒ $i + j > 5$ ☐ nunca puede asegurarse, depende del recorrido

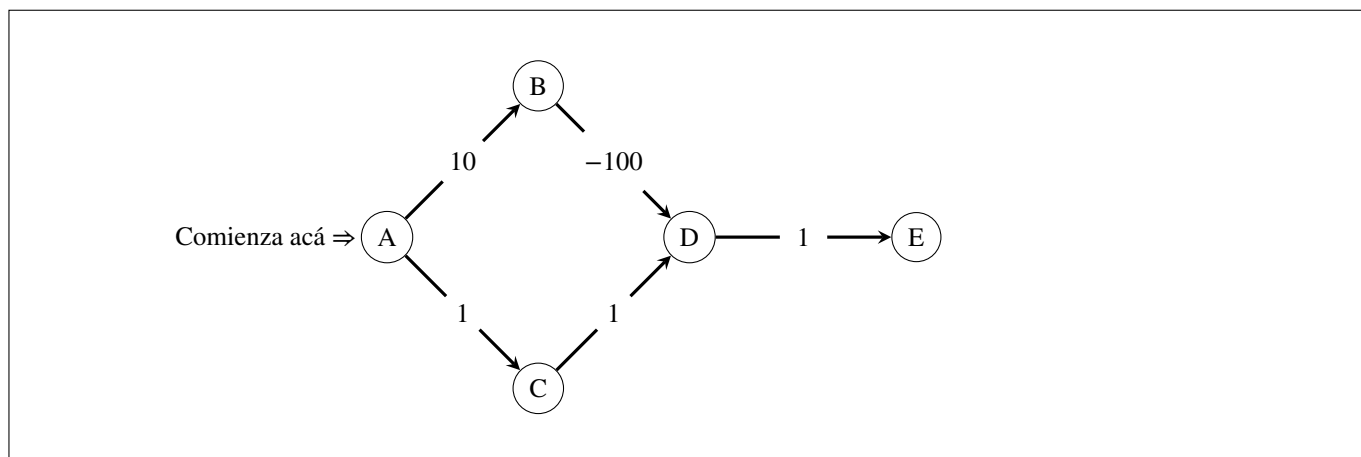
Ejercicio 4: Si una función recursiva tiene complejidad $T(n) = bT(n/b) + \Theta(n)$ con $b > 1$, entonces $T(n)$ es

☐ $\Theta(n)$ ☒ $\Theta(n \log n)$ ☐ $\Theta(n \log^2 n)$ ☐ $\Theta(n^2)$

Ejercicio 5: Dar un ejemplo de un grafo conexo pesado y una arista nueva, tal que agregarle la arista al grafo reduzca el peso del AGM del grafo.



Ejercicio 6: Sabemos que el algoritmo de Dijkstra puede fallar si en un grafo dirigido pesado se tienen pesos negativos. Dar un ejemplo de una instancia (grafo y nodo de origen) donde esto ocurra.



Ejercicio 7: Sea $I = \{[l_1, r_1], [l_2, r_2], \dots, [l_n, r_n]\}$ un conjunto de intervalos sobre la recta real. Queremos cubrir todos los intervalos colocando la menor cantidad de puntos $x_1, \dots, x_k \in \mathbb{R}$, tales que para todo $[l_i, r_i]$ exista algún $x_j \in [l_i, r_i]$. Consideramos el siguiente algoritmo:

- Ordenar los intervalos por su extremo derecho r_i en orden creciente.
- Inicializar el conjunto de puntos $S = \emptyset$
- Para cada intervalo $[l_i, r_i]$, si no existe ningún punto de S que lo cubra, agregamos r_i a S .

Demostrar que este algoritmo devuelve una solución óptima, es decir, que S tiene la mínima cantidad posible de puntos que cubren todos los intervalos.

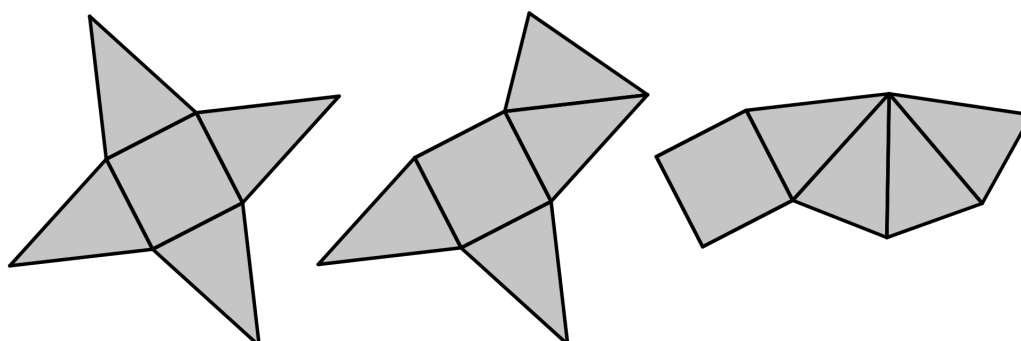
Solución: Sea $G = \{g_1, \dots, g_k\}$ (con $g_1 < g_2 < \dots < g_k$) la solución provista por el greedy. Vamos a demostrar por inducción en i que para todo $i \leq k$, existe una solución óptima que contiene los primeros i elementos de G .

- El caso base $i = 0$ es verdad por vacuosidad: toda solución contiene los primeros 0 elementos de G .
- Paso inductivo ($i > 1$): por hipótesis inductiva, sabemos que existe una solución óptima $S = \{s_1, \dots, s_q\}$ que contiene los primeros $i - 1$ elementos de G , es decir $\{g_1, \dots, g_{i-1}\} \subseteq S$. Queremos, a partir de esta solución, encontrar una solución óptima S' tal que $\{g_1, \dots, g_i\} \subseteq S'$, y con eso habremos demostrado el paso inductivo. Sea $[l, r]$ el intervalo que llevó a que se agregue g_i a G (es decir, $r = g_i$, y además $l > g_{i-1}$, sino g_{i-1} ya hubiese cubierto $[l, r]$), y sea $x \in S$ algún punto que lo cubra. Propongo la solución $S' = (S \setminus \{x\}) \cup \{g_i\}$. Queremos ver tres cosas:
 - Que sigue siendo óptima: esto es así porque por cómo la definimos $|S'| = |S|$, y esta última era óptima.
 - Que $\{g_1, \dots, g_i\} \subseteq S'$: todos los g_j con $j < i$ están en S' pues estaban en S (y son menores o iguales a g_{i-1}), y g_i está en S' por construcción.
 - Que sigue siendo solución: para esto debemos demostrar que no hay ningún intervalo que quede sin cubrir. Supongamos que sí hay algún intervalo $[l', r']$ que no está cubierto. Notemos que $x \in [l', r']$ (pues sino S tampoco lo cubriría), y que $r' < g_i$ y $l' > g_{i-1}$ (pues estos no lo cubren). Pero entonces el intervalo $[l', r']$ es un intervalo que venía antes que $[l, r]$ en el ordenamiento del greedy, y tampoco estaba cubierto con los puntos puestos hasta el momento ($g_1, \dots, g_{i-1}$), entonces el greedy hubiera puesto r' en lugar de $r = g_i$. Contradicción! Entonces queda demostrado.

Entonces, existe una solución óptima que contiene a todo G . Como G en sí es una solución, esto demuestra que G es óptima.

Nota: El greedy produce una solución pues todo intervalo está cubierto: si alguno no lo estaba, cuando lo procesamos agregamos el r_i que lo cubre, y como nunca sacamos elementos del conjunto en el greedy, el resultado final también lo cubre.

Ejercicio 8: Querés armar una pirámide 3D. Para eso, primero tenés que imprimir y recortar un papel, y luego doblar y pegar los bordes expuestos hasta formar la figura 3D. Los lados de la base mide 4cm y las otras aristas, 6cm.

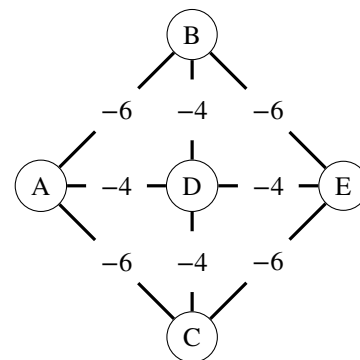


En la ilustración se ven tres ejemplos de cómo podrías armar el recorte. Tenés que encontrar, de todas las formas posibles de armarlo, una que requiera pegar la menor longitud total de bordes para armar la figura 3D. Modelar con el problema de árbol generador mínimo. Esto es, definir un grafo tal que resolver el problema propuesto sea equivalente a encontrar un árbol generador mínimo en ese grafo y **explicar** como recuperar la solución al problema original a partir del AGM del grafo.

Solución: Se considera un grafo donde cada nodo es una cara de la pirámide (hay 5 en total, una para cada cara triangular y una para la base) y un eje entre dos nodos si sus caras correspondientes comparten un lado. El peso de ese eje es el negativo de la longitud de segmento entre esas caras.

La forma de reconstruir es la siguiente: primero tomo un nodo del árbol y lo vuelo la raíz. El polígono que corresponde a ese nodo lo dibujo en el papel. Luego recorro el árbol desde esa raíz (por ejemplo, con DFS), y cada vez que visito un nodo nuevo dibujo su polígono correspondiente en el papel de tal forma que comparta un lado con su padre.

Solución alternativa: Otra solución válida es tomar como nodos los vértices de la pirámide, y de ejes las aristas de la pirámide (donde su peso es la longitud de la arista). En este caso, las aristas del AGM representan qué lados hay que pegar.

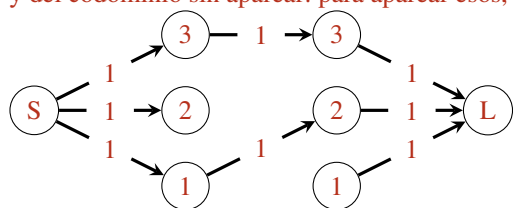


Ejercicio 9: Se tiene un digrafo G con conjunto de nodos $\{1, \dots, n\}$. La *belleza* de una permutación f es la cantidad de índices $i \in \{1, \dots, n\}$ tales que $(i, f(i))$ es un eje del digrafo. Modelar el problema de encontrar la permutación más bella usando max-flow.

Nota: una permutación es una función biyectiva $f : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$.

Solución: Se crea una red de flujo con $2n + 2$ nodos: el nodo de salida y el de llegada, y luego n nodos en la primera capa (estos representan elementos del dominio $\{1, \dots, n\}$) y n nodos en la segunda (estos representan elementos del codominio $\{1, \dots, n\}$). Se agregan aristas de capacidad 1 entre el nodo de salida y todos los nodos de la primera capa, aristas de capacidad 1 entre los nodos de la segunda capa y el de llegada, y finalmente por cada arista (i, j) de G se agrega una arista con capacidad 1 entre el i -ésimo nodo de la primera capa y el j -ésimo de la segunda.

Para recuperar la biyección mas bella, para el i -ésimo nodo de la primera capa, si tiene una arista con flujo positivo que va al j -ésimo nodo de la segunda capa, asignamos $f(i) = j$. Luego de hacer todas esas asignaciones, quedarán elementos del dominio y del codominio sin aparear: para aparear esos, lo hacemos de menos a mayor.



Ejemplo donde $n = 3$ y las aristas de G son $(1, 2)$ y $(3, 3)$

Ejercicio 10: James Bo está atrapado en un grafo de n vértices y m aristas ($m > n$). Para escapar debe ir del vértice en el que se encuentra a alguno de los marcados como “de salida” en a lo sumo t turnos. En cada turno puede recorrer exactamente una arista o permanecer en el nodo en el que está. Hay un problema: en cada turno $1, \dots, t$, puede haber algunas aristas con trampas mortales activadas. El servicio de inteligencia oriental le proveyó con una servilleta que contiene:

- La matriz M de tamaño $m \times t$ que indica si la arista i puede atravesarse en el turno j .
- La estructura del grafo, donde además están señalados tanto el vértice donde él se encuentra prisionero como el conjunto S de vértices de salida.

Dar un algoritmo de complejidad temporal $O(mt)$ para encontrar un camino de escape de la mínima cantidad de turnos, si existiera.

Solución: Sea $G = (V, E)$ el grafo de entrada. Consideremos el grafo $G' = (V', E')$, donde $V' = V \times \{0, \dots, t\}$ (es decir, los nodos de G' son tuplas de nodos de G y tiempos posibles). El conjunto E' se define como unión de dos tipos de arista:

- Aristas de la forma $(i, c) \rightarrow (i, c + 1)$, que corresponden a “esperar” en el nodo i durante el turno c .
- Aristas de la forma $(i, c) \rightarrow (j, c + 1)$, donde $i \rightarrow j$ es una arista de G que se puede atravesar en el turno c .

Notemos que las aristas en G' corresponden a movimientos válidos, y por ende caminos en G' corresponden a secuencias válidas de movimientos. El algoritmo consiste entonces en los siguientes pasos:

- Construir el grafo G' .
- Correr BFS (o cualquier otro algoritmo que determine alcanzabilidad) en G' comenzando en $(I, 0)$, donde I es el nodo donde arranca James Bo.
- Determinar el menor turno posible en el que puedo estar en una salida. Es decir, para cada c en $0, \dots, t$, itero todos los nodos de la forma (s, c) donde $s \in S$ y me fijo si alguno es alcanzable. Ni bien encuentro alguno que lo sea, devuelvo el camino de $(I, 0)$ a (s, c) como resultado. Si no encuentro ninguno, no se puede.

Como el grafo G tiene $O(nt) \subseteq O(mt)$ nodos y $O(nt + mt) \subseteq O(mt)$ aristas, la complejidad es la esperada.